🗼 TOWER DEFENSE

# Soutenance de projet

Conception orientée objet · Java · Semestre 2

Équipe · Livrable Final · Avril 2025

**Modélisation OO**

**Patron de conception**

**Extensibilité**

**Travail d'équipe**

# Plan de la présentation

01

## État d'avancement du projet

Livrables réalisés, tests, documentation

02

## Choix de modélisation

Interfaces, classes abstraites, polymorphisme – pourquoi ces choix ?

03

## Extensibilité

Comment le modèle permet d'ajouter de nouvelles tours, ballons…

04

## Organisation de l'équipe

Répartition des rôles, méthode de travail

05

## Difficultés rencontrées

Défis techniques et de coordination

06

## Bilan & perspectives

Ce que nous retenons du projet

# État d'avancement

L1 — Plateau & Chemins

L2 — Ballons & Déplacement

L3 — Tours & Tir

L4 — Manches & Vagues

L5 — Achat / Vente

LF — Jeu Final

## ⚙️ Développement

- 7 types de tours
- 2 types de plateaux
- Ballons avec effets (gel, ralentissement)
- Système achat/vente
- Joueur interactif et aléatoire

## 🧪 Tests

- Tests unitaires JUnit
- Tests sur Board, Player, Tour, Balloon
- Validation des évolutions
- Tests des chemins

## 📄 Documentation

- Javadoc complète
- README structuré
- 4 fichiers JAR livrés
- Diagrammes UML par livrable

02

# Choix de modélisation – Vue d'ensemble

### «abstract» Balloon

*Classe abstraite*

BasicBalloon en est la seule implémentation actuelle, mais d'autres types (BlinkyBalloon, ArmoredBalloon...) peuvent être ajoutés sans modifier le reste du code.

### «abstract» Tour

*Classe abstraite*

SingeFlechettes, Gorille, TourBombe... héritent tous de Tour. La méthode tirer() est surchargée pour chaque comportement spécifique.

### «abstract» Board

*Classe abstraite*

BoardOne (chemin unique) et BoardTwo (multi-chemins) partagent la même interface. Le plateau délègue la génération de chemins à Path (interface).

Evolution — classe abstraite
Permet d'acheter/vendre des upgrades sur n'importe quelle Tour

02

# Justification des choix de modélisation

### 🔷 Pourquoi Tour est abstraite ?

Chaque tour tire différemment (zone, ciblé, aléatoire, effet spécial). La classe abstraite force l'implémentation de tirer() tout en partageant le scheduler, la portée et les évolutions.
**Gain : ajouter une tour = 1 classe, 1 méthode. Zéro modification du code existant.**

### 🔵 Pourquoi Board est abstraite ?

BoardOne et BoardTwo diffèrent dans la gestion des ballons (1 chemin vs plusieurs). L'abstraction partage move(), removeBalloon(), affiche()... et délègue la spécificité à chaque sous-classe.
**Gain : Game et FinalGame ne connaissent que Board — pas les détails du plateau.**

### 🟩 Pourquoi Path est une interface ?

PathOne (chemin aléatoire en L) et PathTwo (chemin rectiligne) sont deux algorithmes interchangeables. L'interface garantit le contrat findPath() sans imposer de hiérarchie de classes.
**Gain : ajouter un algorithme de chemin = 1 classe qui implémente Path.**

### 🟠 Pourquoi ListChooser est une interface ?

InteractiveListChooser et RandomListChooser proposent deux stratégies de saisie. Le pattern Strategy permet de les injecter dans FinalGame sans modifier la logique du jeu.

**Gain : on peut tester le jeu entier de façon non interactive (random).**

### 🟣 Pourquoi Choice est une interface ?

AcheterTour, VendreTour, AcheterEvolution, VendreEvolution, Jouer... sont autant d'actions homogènes. L'interface Choice impose apply(Player, Board) à chacune, ce qui permet de les stocker dans une liste et de les traiter uniformément.

**Gain : ajouter une action au jeu = 1 classe, sans toucher à FinalGame.**

03

# Extensibilité du modèle

| Extension souhaitée | Comment ? | Impact sur le code existant |
|---|---|---|
| ➕ Une nouvelle tour | Créer une classe extends Tour, surcharger tirer() | **0 modification** |
| ➕ Un nouveau type de ballon | Créer une classe extends Balloon | **0 modification** |
| ➕ Un algorithme de chemin | Implémenter l'interface Path | **0 modification** |
| ➕ Une nouvelle évolution | Créer une classe extends Evolution | **0 modification** |
| ➕ Un nouveau plateau | Créer une classe extends Board | **0 modification** |
| ➕ Un mode de jeu | Créer une classe extends Game, surcharger play() | **0 modification** |

04

# Organisation de l'équipe

## 🤝 Méthode de travail

- Travail livrable par livrable
- Réunions de synchronisation régulières
- Code review avant merge
- Pair-programming sur les parties complexes
- Git : branches par fonctionnalité
- Rôles tournants (chef de projet, intégrateur)

## 📋 Répartition des tâches

| | |
|---|---|
| Binôme A | Board & Chemins |
| Binôme B | Balloon & Déplacement |
| Binôme A | Tours & ScheduledExecutor |
| Binôme B | Player & Économie |
| Ensemble | FinalGame & CLI |
| Ensemble | Tests & Documentation |

05

# Difficultés rencontrées

## 💥 Supprimer un ballon après destruction

⚠️ **Problème :** Quand un ballon tombait à 0 PV, il restait dans la liste du board. Les tours continuaient à le cibler et à lui infliger des dégâts → comportement incohérent.

✅ **Solution :** Après chaque takeDamage(), on appelle removeBalloon() immédiatement si isDestroyed() est vrai — dans generate() et dans le game loop.

## 🏗️ Placer une tour sur une cellule occupée

⚠️ **Problème :** Rien n'empêchait d'ajouter une tour sur une cellule de chemin, là où passent les ballons. La tour écrasait le chemin et les ballons n'avaient plus de case pour avancer.

✅ **Solution :** putTower() vérifie d'abord que la position ne fait pas partie de la liste path avant de placer la tour. Retour false si invalide.

## 🔁 Ballons plus nombreux que les chemins

⚠️ **Problème :** Avec BoardTwo, chaque ballon reçoit un chemin. Quand il y a plus de ballons que de chemins, l'accès à l'index dépassait la taille de la liste → IndexOutOfBoundsException.

✅ **Solution :** Affectation cyclique dans addBalloon() : index modulo nombre de chemins. Chaque chemin est réutilisé équitablement.

## 🤝 Coordination & conflits Git

⚠️ **Problème :** (Nous expliquerons oralement les détails).

✅ **Solution :** Découpage en branches par fonctionnalité, revues avant merge, réunions courtes de synchronisation.

# Bilan & perspectives

## ✅ Ce que nous avons appris

- Maîtrise des classes abstraites et interfaces Java
- Compréhension concrète du polymorphisme
- Gestion de la concurrence (threads, ScheduledExecutor)
- Travail en équipe avec Git / code review
- Conception extensible : ouvert à l'extension, fermé à la modification

## 🚀 Perspectives d'extension

- Interface graphique (JavaFX ou Swing)
- Nouveaux types de ballons (blindés, régénérants…)
- Système de score et sauvegarde
- Niveaux de difficulté progressifs



# Merci de votre attention

*Questions ?*